

Prevenção de Problemas de Configuração e Build

Data de criação: 13/12/2025

Baseado em: problemas identificados e corrigidos durante o setup do projeto **Aurora Platform**

Objetivo

Este documento estabelece práticas e ferramentas para **prevenir** que problemas de configuração, dependências e build voltem a ocorrer em futuras implementações ou atualizações.

Sumário

1. Validação de Dependências
 2. Testes de Inicialização Automatizados
 3. Scripts de Build Limpo
 4. Validação de Configuração TypeScript
 5. Health Check Automatizado
 6. Validação de Versões Node em Dockerfiles
 7. Pre-commit Hooks
 8. Documentação de Breaking Changes
 9. CI/CD Pipeline
 10. Checklist de Nova Feature/Serviço
-

1. Validação de Dependências

1.1 Lock de Versões Críticas

Problema evitado: breaking changes em dependências (ex.: `@nestjs/throttler` v5 → v6)

Solução: - Criar arquivo `.nvmrc`. - Definir `engines` no `package.json`. - Fixar versões exatas para dependências críticas.

Exemplo (raiz do repositório ou padrão adotado por pacote):

```
{
  "engines": {
    "node": ">=20.0.0",
    "npm": ">=10.0.0"
  },
  "dependencies": {
    "@nestjs/core": "11.1.9",
    "@nestjs/throttler": "6.0.0"
  }
}
```

```
}  
}
```

Script de validação (`scripts/validate-deps.sh`)

```
#!/bin/bash  
# Valida compatibilidade de versões entre serviços  
  
SERVICES=("auth-service" "users-service" "events-service" "registrations-  
service")  
CRITICAL_DEPS("@nestjs/core" "@nestjs/throttler" "@nestjs/typeorm")  
  
for service in "${SERVICES[@]}; do  
  echo "Validando $service..."  
  for dep in "${CRITICAL_DEPS[@]}; do  
    version=$(jq -r ".dependencies[\"$dep\"]" "packages/$service/  
package.json")  
    echo "  $dep: $version"  
  done  
done
```

Executar antes de commit:

```
chmod +x scripts/validate-deps.sh  
./scripts/validate-deps.sh
```

2. Testes de Inicialização Automatizados

2.1 Smoke Tests para Cada Serviço

Problema evitado: serviços com erro de configuração que crasham ao iniciar (`ThrottlerGuard`, `crypto`, `paths`)

Solução: Criar testes que verificam **apenas se o serviço inicia sem erro**.

Arquivo: `packages/users-service/test/smoke.spec.ts`

```
import { Test } from '@nestjs/testing';  
import { AppModule } from '../src/app.module';  
  
describe('Service Smoke Test', () => {  
  it('should initialize application without errors', async () => {  
    const moduleRef = await Test.createTestingModule({  
      imports: [AppModule],  
    }).compile();  
  });  
});
```

```

    const app = moduleRef.createNestApplication();
    await expect(app.init()).resolves.not.toThrow();
    await app.close();
  }, 30000);

it('should have ThrottlerGuard configured correctly', async () => {
  const moduleRef = await Test.createTestingModule({
    imports: [AppModule],
  }).compile();

  const app = moduleRef.createNestApplication();
  await app.init();

  // Verifica se ThrottlerGuard está registrado
  const guards = Reflect.getMetadata('__guards__', AppModule) || [];
  expect(guards.length).toBeGreaterThan(0);

  await app.close();
});
});

```

Executar em CI:

```
npm run test:smoke --prefix packages/users-service
```

3. Scripts de Build Limpo

3.1 Clean Build Script

Problema evitado: Docker usando código compilado antigo devido a cache incremental do TypeScript

Solução: Adicionar script `clean:build` em **todos** os `package.json`.

```

{
  "scripts": {
    "clean": "rm -rf dist tsconfig.tsbuildinfo",
    "clean:build": "npm run clean && npm run build",
    "docker:clean":
      "rm -rf dist tsconfig.tsbuildinfo && docker compose build --no-cache"
  }
}

```

Uso:

```
# Build local limpo
npm run clean:build

# Rebuild Docker limpo
npm run docker:clean
```

4. Validação de Configuração TypeScript

4.1 Script de Validação de Paths

Problema evitado: `rootDir` no tsconfig gerando paths incorretos no `dist/`

Solução: Criar script que valida consistência entre `tsconfig.json` e `package.json`.

Arquivo: `scripts/validate-tsconfig.js`

```
const fs = require('fs');
const path = require('path');

const services = ['auth-service', 'users-service', 'events-service',
  'registrations-service'];

services.forEach(service => {
  const tsconfigPath = `packages/${service}/tsconfig.json`;
  const packageJsonPath = `packages/${service}/package.json`;

  const tsconfig = JSON.parse(fs.readFileSync(tsconfigPath, 'utf8'));
  const packageJson = JSON.parse(fs.readFileSync(packageJsonPath, 'utf8'));

  const rootDir = tsconfig.compilerOptions?.rootDir || '.';
  const outDir = tsconfig.compilerOptions?.outDir || 'dist';

  // Calcula o caminho esperado baseado no rootDir
  let expectedPath;
  if (rootDir === '..') {
    expectedPath = `dist/app/src/main.js`;
  } else {
    expectedPath = `dist/src/main.js`;
  }

  const actualPath = packageJson.scripts['start:prod']?.split('node ')[1];

  if (actualPath !== expectedPath) {
    console.error(`❌ ${service}: Inconsistência detectada!`);
    console.error(`  rootDir: ${rootDir}, outDir: ${outDir}`);
    console.error(`  Esperado: ${expectedPath}`);
    console.error(`  Atual: ${actualPath}`);
  }
});
```

```

        process.exit(1);
    } else {
        console.log(`✅ ${service}: Paths consistentes`);
    }
});

console.log(`\n✅ Todos os serviços validados com sucesso!`);

```

Executar:

```
node scripts/validate-tsconfig.js
```

5. Health Check Automatizado

5.1 Script de Validação Pós-Deploy

Problema evitado: deploy com serviços que não respondem corretamente

Solução:

Arquivo: `scripts/health-check.sh`

```

#!/bin/bash
set -e

SERVICES=(
    "users-service:3011:/users/health"
    "auth-service:3010:/health"
    "events-service:3012:/health"
    "registrations-service:3013:/registrations/health"
)

echo "🚨 Executando health checks..."

for service_config in "${SERVICES[@]}; do
    IFS=':' read -r name port path <<< "$service_config"

    echo "Testando $name em http://localhost:$port$path..."

    response=$(curl -s -o /dev/null -w "%{http_code}" "http://localhost:
$port$path" || echo "000")

    if [ "$response" = "200" ]; then
        echo "✅ $name: OK"
    else
        echo "❌ $name: FALHOU (HTTP $response)"
    fi
done
exit 1

```

```

    fi
done

echo ""
echo "✅ Todos os serviços passaram no health check!"

```

Executar após `docker compose up`:

```

chmod +x scripts/health-check.sh
sleep 15 && ./scripts/health-check.sh

```

6. Validação de Versões Node em Dockerfiles

6.1 Linter para Dockerfiles

Problema evitado: inconsistência de versões Node entre serviços (Node 18 vs 20)

Solução:

Arquivo: `scripts/validate-dockerfiles.sh`

```

#!/bin/bash

EXPECTED_NODE_VERSION="20"

echo "🔍 Validando versões do Node nos Dockerfiles..."

for dockerfile in packages/*/Dockerfile; do
    service=$(dirname "$dockerfile" | xargs basename)

    # Extraí versão do Node
    node_version=$(grep -E "FROM node:" "$dockerfile" | head -1 | sed -E
's/.*node:([0-9]+).*/\1/')

    if [ "$node_version" != "$EXPECTED_NODE_VERSION" ]; then
        echo "❌ $service: Node $node_version (esperado: $EXPECTED_NODE_VERSION)"
        echo "  Arquivo: $dockerfile"
        exit 1
    else
        echo "✅ $service: Node $node_version"
    fi
done

echo ""
echo "✅ Todos os Dockerfiles validados!"

```

7. Pre-commit Hooks

7.1 Husky + Validações

Problema evitado: commits com configurações quebradas

Solução: Instalar Husky na raiz do projeto:

```
npm install --save-dev husky
npx husky init
```

Arquivo: `.husky/pre-commit`

```
#!/bin/sh
. "$(dirname "$0")/_/husky.sh"

echo "⚠ Executando validações pré-commit..."

# Valida dependências
./scripts/validate-deps.sh || exit 1

# Valida tsconfig paths
node scripts/validate-tsconfig.js || exit 1

# Valida Dockerfiles
./scripts/validate-dockerfiles.sh || exit 1

echo "✅ Validações concluídas com sucesso!"
```

8. Documentação de Breaking Changes

8.1 Changelog de Dependências

Criar arquivo `DEPENDENCY_UPDATES.md` para documentar **mudanças que exigem ação**.

```
# Histórico de Atualizações de Dependências

## @nestjs/throttler: 5.x → 6.0.0

**Data:** 13/12/2025
**Breaking Change:** ✅ SIM

**O que mudou:**
- Configuração agora exige array `throttlers`
- TTL passou de segundos para milissegundos
```

****Ação necessária:****

```
```typescript
// ANTES
ThrottlerModule.forRoot({ ttl: 60, limit: 100 })

// DEPOIS
ThrottlerModule.forRoot({ throttlers: [{ ttl: 60000, limit: 100 }] })
```

**Arquivos afetados:** - Todos os `app.module.ts` com ThrottlerModule

```

9. CI/CD Pipeline

9.1 GitHub Actions Workflow
Arquivo: `.github/workflows/validate.yml`

```yaml
name: Validate Configuration

on:
  pull_request:
    branches: [main, develop]
  push:
    branches: [main]

jobs:
  validate:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '20'

      - name: Validate dependencies
        run: |
          chmod +x scripts/validate-deps.sh
          ./scripts/validate-deps.sh

      - name: Validate TypeScript configs
        run: node scripts/validate-tsconfig.js

      - name: Validate Dockerfiles
        run: |
          chmod +x scripts/validate-dockerfiles.sh
```



```

        ./scripts/validate-dockerfiles.sh

- name: Run smoke tests
  run: |
    for service in auth-service users-service events-service
registrations-service; do
      echo "Testing $service..."
      npm install --prefix packages/$service
      npm run test:smoke --prefix packages/$service
    done

- name: Build Docker images
  run: docker compose -f docker-compose.dev.yml build

- name: Start services
  run: docker compose -f docker-compose.dev.yml up -d

- name: Wait for services
  run: sleep 30

- name: Health checks
  run: |
    chmod +x scripts/health-check.sh
    ./scripts/health-check.sh

- name: Cleanup
  if: always()
  run: docker compose -f docker-compose.dev.yml down -v

```

10. Checklist de Nova Feature/Serviço

Quando adicionar novo serviço ou atualizar dependências:

✓ Checklist Obrigatório

- [] Versão Node >= 20 no Dockerfile
- [] ThrottlerModule configurado com array `throttlers` (v6+)
- [] ConfigModule com validação Joi para todas as env vars
- [] `search_path` inclui `public` no TypeORM config
- [] Polyfill crypto se Node < 20
- [] Scripts `start` e `start:prod` apontam para path correto do `dist/`
- [] Smoke test criado em `test/smoke.spec.ts`
- [] Health endpoint implementado
- [] Adicionado ao `scripts/health-check.sh`
- [] Adicionado ao `scripts/validate-deps.sh`
- [] `package.json` tem script `clean:build`
- [] Documentado em `DEPENDENCY_UPDATES.md` se houver breaking change

Resumo

Problemas prevenidos com estas práticas:

Problema	Prevenção
Breaking changes em deps	Lock de versões + validação
Serviços que crasham ao iniciar	Smoke tests + CI/CD
Docker com código antigo	Scripts clean build
Paths TypeScript incorretos	Validação tsconfig
Inconsistência Node entre serviços	Validação Dockerfiles
Deploy com serviços quebrados	Health checks automatizados
Commits com configurações ruins	Pre-commit hooks

Tempo estimado de setup inicial: ~2 horas

Tempo economizado em debugging futuro: inestimável